

### Análise Quantitativa

- RQ1: Qual é a acurácia das LLMs ao julgar qualidade de código, em comparação com um avaliador humano?

ChatGPT - 24/38

Llama - 21/38

- RQ3: Quando as LLMs e o avaliador humano tem o mesmo desempenho?

LLMs igual ao humano: 14/38

### Análise Qualitativa

- RQ2: Quais fatores contribuem para as divergências entre os julgamentos das LLMs e de avaliadores humanos sobre qualidade de código?

ambas lms abandonaram - 7 dos casos

Chat GPT: 14 abandonados

Llama: 17 abandonados

## RQ2 - Análises

**12150\_rev1:** as LLMs divergiram por questões de **arquitetura do código**

- chatGPT: em questões de estrutura de código, se remover o if (TestConfig.isMembase()), o código perde em questões de manutenção e teste, pois em caso de mudanças de ambientes, pode ocorrer falha
- Llama: considerou que a repetição do TestConfig.isMembase() poderia causar problemas de manutenibilidade, caso houvesse uma mudança que deixasse a condição mais complexa

**12285\_rev1:** a LLM deu diferente por questões de **falta contexto externo**

- chatGPT: apesar de achar o Embora o Código 2 seja mais conciso, essa concisão tem o custo da segurança e da legibilidade potencial quando as suposições não são documentadas.
- Llama: o princípio de single responsibility é ferido no Code1, pois a função faz mais de uma coisa, por isso classificou o código 2 como o melhor.

**13082\_rev1: erro de análise do Llama**

- **chatGPT:** Considerou que ter o + de concatenação no início da frase é uma melhor prática
- Llama: comentou faltas do code 2, mas que não são verdades. O código melhorado que ele sugeriu é o code 2, mas abandonou a PR =S.
- humano: corrigindo issues de checkstyle

**15955\_rev1: erro de análise do Llama**

- **chatGPT:** Acertou na análise, conforme dito no PR pelo humano, que tem redundância no código que foi removida, deixando o código mais conciso.
- Llama: A LLM não compreendeu o objetivo da mudança, não observando que todos os IFs fazem a mesma coisa e que o *stale* é uma classe e não uma variável / string
- Humano: Stale é redundante, por isso foi removido.

**15956\_rev1:** o Llama deu diferente por questões de **falta contexto externo**

- **chatGPT:** Compreendeu o objetivo de melhor manutenção e clareza do código.
- Llama: Detectou uma relação inexistente entre os parâmetros, além de falar que o code 2 não segue os padrões de código (cameCase)
- Humano: A função setGroup(boolean group, int group\_level) foi removida por ser redundante, já que o parâmetro groupsobrescrevia group\_level. No lugar, foi criada a função setGroupLevel(), que simplifica e torna a configuração mais direta.

**18807\_rev1:** o Llama deu diferente por questões de **falta contexto externo**

- **chatGPT:** conseguiu fazer uma análise verificando a possibilidade do código trabalhar com multithread. Presumiu que está trabalhando com multithread devido às mudanças, por isso avaliou como “merged”
- Llama: conseguiu fazer uma análise verificando a possibilidade do código trabalhar com multithread. Não presumiu que está trabalhando com multithread devido às mudanças, por isso avaliou como “abandoned”
- Humano: Por ser multithread, fez a modificação

**30045\_rev1:** o Llama considerou melhor o padrão de código, enquanto o GPT “olhou” mais para questões de manutenibilidade

- chatGPT: Considerou a primeira versão melhor em questão de adaptação à mudança da mensagem, mas sugeriu melhorias ao utilizar componentes do código 2
- Llama: por questões de padrão de código, considerou o code2 melhor

**33930\_rev1:** as LLMs divergiram por questões de **arquitetura do código**

- chatGPT: recusou porque fere questões de clareza de código
- Llama: achou o code2 mais fácil de entender e dar manutenção

**34885\_rev1:** as LLMs deram diferente por questões de **falta contexto externo**, pois não dá para julgar a relevância da condição sem contexto completo

- **chatGPT:** achou válida as mudanças, principalmente por checar mais condições
- Llama: achou inválido as mudanças, pois achou redundante if (alternative == null || !alternative.isActive())
- Humano: O tratamento de erros NMVB foi melhorado: o conjunto de nós inválidos agora é esvaziado quando cheio, e um check de configuração é acionado para corrigir setups desatualizados. Isso torna o sistema mais robusto e responsivo a falhas.

**41484\_rev1:** as LLMs divergiram por questões de **arquitetura do código**

- **chatGPT:** Embora o código tenha mais linhas devido aos comentários, ele melhora a manutenibilidade
- Llama: Achou desnecessário a inclusão dos comentários de copyright e dados de licença, sugerindo a remoção.
- Humano: Adicionou mais linhas devido aos comentários de copyright e dados de licença

**48165\_rev1:** as LLMs deram diferente por questões de **falta contexto externo**, pois não dá para julgar a relevância da condição sem contexto completo

- ChatGPT: Embora o Código 2 seja visualmente mais limpo por ter menos linhas e evitar o uso de try-catch, o Código 1 é mais explícito quanto aos tipos esperados, o que facilita o entendimento, especialmente para quem não conhece bem a API usada. Além disso, sua estrutura defensiva, como o fallback de int para long, mostra maior atenção a casos de erro, o que contribui para uma manutenção mais segura e precisa. Assim, apesar de mais verboso, o Código 1 oferece maior robustez e é recomendado como a melhor versão.
- Llama: O Código 2 foi considerado ligeiramente superior ao Código 1 em termos de clareza, legibilidade e manutenibilidade. Ele utiliza métodos booleanos mais diretos, remove blocos desnecessários como try-catch e elimina castings redundantes, o que torna a lógica mais simples e o código mais fácil de entender e manter
- Humano: Melhorar a performance da decodificação de valores long, que anteriormente envolvia um fallback lento baseado em exceções ao tentar converter de int para long. A modificação substitui essa abordagem pelo uso direto dos métodos do parser Jackson, que já determinam o tipo numérico automaticamente, tornando o código mais simples e eficiente. Além disso, foram adotados métodos de acesso direto mais rápidos, eliminando conversões desnecessárias. Como resultado, a decodificação de valores long tornou-se significativamente mais rápida.

**50963\_rev1:** a LLM deu diferente por questões de **falta contexto externo**

- chatGpt: detectou que não há mudança de lógica, só de documentação e alegou que a mudança deixou a documentação do código superior
- Llama: alegou que o comentário (allBut()) pode ser difícil para pessoas que não conhecem a biblioteca do CouchBase entender
- humano: As modificações incluíram a atualização do Javadoc da classe para destacar corretamente os métodos estáticos anyOf, allBut e any como pontos de entrada. Além disso, foi documentado o comportamento do sistema quando os erros persistem além do número máximo de tentativas permitido (maxAttempts), especificando que, nesse caso, uma exceção CannotRetryException é lançada. Essa explicação adicional foi incluída tanto na documentação da classe quanto nos métodos max e once.

**51210\_rev1:** as LLMs divergiram por questões de **arquitetura do código**

- chatGPT: O Código 2 é superior em termos de manutenibilidade e clareza de intenção, mesmo que o Código 1 seja mais minimalista.
- Llama: O Código 1 é preferível por ser mais claro, legível e fácil de manter. Ele é direto e conciso, enquanto o Código 2 adiciona complexidade desnecessária com verificações. Ambos podem ser aprimorados, mas o Código 1 se destaca pela simplicidade. Pelo Llama a checagem do llama é desnecessária.
- humano: A motivação da mudança foi garantir que o método query(Statement) funcione não apenas como atalho para consultas simples, mas também seja compatível com objetos do tipo QueryPlan.

**59291\_rev1:** a LLM deu diferente por questões de **falta contexto externo**

- **ChatGPT:** Alega que para o Java8, o código anterior é melhor, dado que String.join() é mais declarativo e compreensível, seguindo o “padrão” da linguagem. Já no segundo, eles criaram um padrão manual de checagem.

- **Llama:** O código mergeado é mais claro, fácil de ler e de dar manutenção, trazendo benefícios na estrutura de código.
- **humano:** join is available only from java 8, so using just append

**61569\_rev1:** a LLM deu diferente por questões de **falta contexto externo**

- **chatgpt:** O Código 1 é preferido por sua clareza, estrutura direta e facilidade de manutenção. Ele realiza uma verificação exata baseada no tamanho em bytes, condizente com a mensagem de erro. Já o Código 2 sacrifica precisão por desempenho, usando length() como pré-validação leve, o que pode confundir sem contexto adequado. O Código 2 só é recomendável em cenários críticos de performance e quando há validação rigorosa garantida em outra camada.
- **Llama:** Code 2 é preferido por ser mais claro, legível e fácil de manter. Ele usa length() em vez de getBytes().length, evitando complexidades com codificação de caracteres e alocação de bytes. A estrutura é mais concisa, o que facilita futuras modificações e reduz chances de erro.
- **humano:** A verificação de comprimento da chave foi simplificada usando String.length() em vez de getBytes(), reduzindo custo e lixo de memória. A verificação precisa continua sendo feita na core-io.

**82684\_rev1:** as LLMs divergiram por questões de **arquitetura do código**

- Llama: No geral, eu classificaria o Código 1 um pouco mais alto do que o Código 2 devido à sua explicitude e aos potenciais problemas com suposições sobre o estado inicial das `métricas`. No entanto, ambos os códigos são relativamente claros e legíveis.
- chatGPT: O Código 2 melhora a clareza, a segurança de tipos, a detecção de erros e a extensibilidade tornando-o a melhor opção no geral.
- Humano: Estenda SendHookTest para acessar opaco.

## RQ3 - Análises

**12150\_rev1:** a LLM deu diferente por questões de **falta contexto externo**

- chatGPT: em questões de estrutura de código, se remover o if (TestConfig.isMembase()), o código perde em questões de manutenção e teste, pois em caso de mudanças de ambientes, pode ocorrer falha
- humano: Este cliente deve sempre trabalhar com Membase e Couchbase, então não precisamos verificar o tipo de servidor. - deu o contexto da remoção

**12285\_rev1:** a LLM deu diferente por questões de **falta contexto externo**

- chatGPT: apesar de achar o Embora o Código 2 seja mais conciso, essa concisão tem o custo da segurança e da legibilidade potencial quando as suposições não são documentadas.
- humano: contexto externo da remoção

**30045\_rev1:** a LLM deu diferente por questões de **falta contexto externo**

- chatGPT: Considerou a primeira versão melhor em questão de adaptação à mudança da mensagem, mas sugeriu melhorias ao utilizar componentes do código 2
- humano: Adaptando o caso de teste para as últimas mudanças de observação.

**33930\_rev1:** a LLM deu diferente por questões de **falta contexto externo**

- chatGPT: recusou porque fere questões de clareza de código
- humano: Reescrevendo os componentes internos da consulta para melhor desempenho

**41484\_rev1: erro de análise do Llama**

- chatGPT: Embora o código tenha mais linhas devido aos comentários, ele melhora a manutenibilidade
- Llama: Achou desnecessário a inclusão dos comentários de copyright e dados de licença, sugerindo a remoção.